DarkGuard AI: System Architecture (v3, Hackathon Scope)

1. Design principle

DarkGuard is **extension-first** and **hybrid in detection**. Three roles are kept separate:

Role	Who does it	Why
Propose: estimate which text may be deceptive	Text classifier (ML)	Wording varies too much for hand-written rules
Verify: confirm concrete evidence	Rule detectors	Timers, arithmetic, and checkbox state are exact and testable
Change: alter what the user sees	Approved templates (fixed catalogue)	Keeps the page trustworthy and every change reversible

No model writes page code, CSS, or content. The backend, when present, only rewrites explanation text.

2. Components

Component	Runs in	Responsibility
Landing page	Vercel (static)	Product description, install guide, demo video, test site link
Popup	Extension	Scan button, results, risk score, history, Ethical View toggle, model status
Service worker	Extension background	Coordinates scans, requests host permission, stores history
Content script	Extension page context	Extracts candidates, runs rules and classifier, applies Ethical View
Scanner	Content script	Builds candidate list (visible elements, no input values)
Rule detectors	Content script	DP-1 to DP-4 evidence checks
Text classifier	Content script	Loads JSON model, verifies checksum, returns class probabilities
Fusion	Content script	Combines rule evidence and classifier output into detections
Risk engine	Content script	Computes 0–100 score and level
Ethical View engine	Content script	Applies and restores templates from the lookup catalogue
API client	Service worker	Optional backend calls, with timeout and fallback
Explain service	Backend (optional)	Rewrites explanation text with an LLM
Training pipeline	Developer machine (`ml/`)	Trains, evaluates, exports model, runs release gate
Test site	Static host	Planted and clean shopping pages
Eval runner	Developer machine (`eval/`)	Rules-only vs rules-plus-ML report

3. Architecture diagram

```
flowchart LR
    subgraph Browser["Browser (all detection happens here)"]
        P[Popup] -->|scan request| SW[Service Worker]
        SW -->|inject| CS[Content Script]
        CS --> SC[Scanner]
        SC --> RD[Rule Detectors DP-1..DP-4]
        SC --> TC[Text Classifier]
        RD --> FU[Fusion]
        TC --> FU
        FU --> RE[Risk Engine]
        RE --> EV[Ethical View Engine]
        EV --> T[(templates.json lookup)]
        CS -->|results| P
```

```
    SW <-->|history| ST[(chrome.storage.local)]
    SW -.optional.-> AC[API Client]
end

AC -.HTTPS, detection summaries only.-> BE[Explain Service FastAPI]
BE -.-> LLM[Hosted LLM]

subgraph Offline["Developer machine (ml/)"]
    D[(Labeled strings)] --> TR[train.py]
    TR --> EX[evaluate.py release gate]
    EX --> MA[(shaming-v1.json + checksum)]
end
MA -.bundled into extension.-> TC
```

Solid lines run on every scan. Dashed lines are optional. The offline pipeline runs before release, not during a scan.

4. Scan flow

1 The user opens a product page and clicks **Scan this page**.
2 If the origin is new, the service worker requests host permission. The user approves.
3 The service worker injects the content script.
4 The scanner builds candidates (visible elements, no input values).
5 Rule detectors run. DP-1 waits for the 10-second observation window.
6 The classifier scores text candidates in under 5 ms. It runs only after its checksum is verified. If verification fails, the scan continues with rules alone.
7 Fusion merges rule evidence and classifier output (PRD FR-13 and TRD section 6.4). Classifier-only signals are capped at confidence 0.5.
8 The risk engine computes the score.
9 Explanations come from templates, so results appear immediately.
10 If "Improve explanations" is on and the backend responds within 3 seconds, the text is replaced. Otherwise the template text stays.
11 Results go to the popup and the scan is saved to history, including the model version.
12 If Ethical View is on, the engine applies templates from the lookup table, including the banner with "Show original."

Timing. A full scan takes about 11 seconds, because of the DP-1 observation window. The popup shows the other results immediately and marks DP-1 as "checking timer…".

5. Model lifecycle

```
Label strings  ->  Train  ->  Evaluate (validation thresholds, test metrics, ablation)
                                  |
                        release gate passes?
                       /                    \
                     yes                     no
                      |                       |
     export JSON + checksum            ship with ML disabled,
     write model card                  report reason
                      |
     bundle in extension  ->  load at scan time (checksum check)
```

Each shipped model has a version ID, a training data hash, a model card, and the gate result. The scan record stores the version so results can be traced later.

6. Ethical View state machine

```
OFF ──apply──▶ ON
 ▲              │
 └──restore─────┘
```

- **OFF:** Page is untouched. The DOM matches the pre-scan state.
- **ON:** Ethical View nodes exist, all tagged `data-dg-owner="ethical-view"`.

- **Failure:** If apply throws partway, restore runs automatically.

7. Data flow and privacy boundaries

Data	Stays in browser	Sent to backend	Stored
Page HTML	Yes	Never	No
Candidate text used for detection	Yes	Never	In memory for the scan
Input values, passwords, payment fields	Never read	Never	Never
Classifier scores	Yes	Never	Summary only (category, probability, model version)
Detection summaries (category, rule, snippet ≤ 200 chars)	Yes	Only if AI explanations are on	Local history
Domain and path	Yes	Domain only, if AI explanations are on	Local history
Training strings	Developer machine	Never	`ml/data/` (short UI strings only, labeled)

8. Permissions boundary

- Access is granted per origin, when the user scans it.
- The service worker is the only component that talks to the network.
- The content script cannot reach the backend. All network calls go through the service worker.
- The model is bundled with the extension. It is not fetched at runtime, so there is no remote code path.

9. Failure modes

Failure	Behavior	User sees
Classifier file missing or checksum mismatch	Rules-only scan	"Text model unavailable. Rule results only."
Classifier slow (over budget)	Scan skips classifier for remaining candidates	"Partial text analysis."
Rule detector throws	That detector is skipped, others continue	Detector listed as "not run"
Backend down or slow	Template explanations used	Nothing extra
Ethical View apply error	Automatic restore	"Could not apply view. Page restored."
Site changes DOM mid-scan	Missing references are skipped	Evidence marked "no longer on page"

Detection never depends on the backend or on the classifier. Each can fail without stopping the scan.

10. Backend (optional)

- FastAPI, one process, two routes: `GET /v1/health`, `POST /v1/explain`.
- Deployed on Render or a small AWS instance. Used only if AI explanations are demonstrated.
- The backend does not run the classifier.
- Not included: database, accounts, scan storage, report hosting.

11. Deployment

Part	Where	Notes
Landing page	Vercel	Static React + Tailwind build
Test site	Vercel or GitHub Pages	Used in demo and evaluation
Extension	Load unpacked for the demo; Chrome Web Store later	Store review takes time, so the hackathon does not depend on it
Model artifact	Bundled in the extension package	Versioned, checksummed
Backend	Render (optional)	Only if AI explanations are demonstrated
Source	GitHub monorepo	Layout in TRD section 3

12. Key architecture decisions

#	Decision	Reason	Trade-off
ADR-1	Detection runs in the browser	Fast, private, works offline	Model must be small enough to bundle
ADR-2	ML classifies text only; rules hold the evidence	Keeps detections explainable and testable	Classifier cannot create strong findings alone
ADR-3	Logistic regression on TF-IDF as the baseline	Small, auditable, fast in JavaScript	Less accurate than transformers on paraphrase
ADR-4	Corrections come from a fixed template lookup, not a model	Prevents unsafe or unexpected page changes	New correction types need a template update
ADR-5	No generative model changes page content	Protects prices, controls, and trust	Less dramatic before-and-after effect
ADR-6	Release gate blocks models that do not beat rules	Honest evaluation, no inflated claims	A weak model ships disabled, which the report must explain
ADR-7	Model integrity checked with SHA-256 on load	Detects tampering and corrupted files	Requires updating the checksum on each release
ADR-8	Per-origin host permission	Least privilege	One extra prompt per new site
ADR-9	No database for the hackathon	Fewer failure points	No cross-device history
ADR-10	Four detection categories	Each can be tested well in the time available	Less coverage than a full dark-pattern taxonomy

13. Folder structure

See TRD section 3. Extension, landing page, test site, optional backend, training pipeline, and evaluation live in one monorepo, so judges can reproduce results from one place.

14. Future architecture

- Transformer or sentence-embedding classifier, if the baseline's recall is not enough and the bundle size allows it.
- Multi-page flows (cart and checkout comparison) with a short-lived local state store.
- Per-site drift monitoring, to detect when a retrained model starts failing on new sites.
- Optional server-side history and a website-owner dashboard, behind accounts.
- Firefox and Safari builds from the same content-script code.